Dictionaries: counting words

Making a dictionary, looking up a key, adding and removing pairs, which keys are allowed, and looping over pairs.

THE QUESTION

How can a program count every word in a text, and find any word's count without searching?

THE INTUITION

A dictionary stores pairs: a key, such as a word, and its value, such as its count. Python goes straight from a key to its value, without searching.

KEY TAKEAWAY

Keys are unique and hashable. `d[k]` fails on a missing key; `d.get(k, 0)` does not. Count with `d[w] = d.get(w, 0) + 1`.

1 A dictionary maps keys to values

menu.py

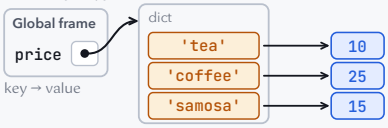
```
price = {"tea": 10, "coffee": 25, "samosa": 15}
print(price)
print(price["coffee"], len(price)) → 25 3
```

line 2 → `{'tea': 10, 'coffee': 25, 'samosa': 15}`

as a table: one row per pair

key	value
'tea'	10
'coffee'	25
'samosa'	15

in memory, as pythontutor.com draws it



A **dictionary** (type `dict`) holds **key-value pairs**: each **key** leads to one **value**, as a word in a printed dictionary leads to its meaning. Write the pairs in braces, a colon inside each pair, commas between. `price["coffee"]` gives the value of the key `"coffee"`; `len` counts pairs. A list finds an item by position, a dictionary by key.

2 Start empty, or zip two lists together

make.py

```
empty = {}
print(empty, len(empty), type(empty)) → {} 0 <class 'dict'>
names = ["tea", "coffee", "samosa"]
costs = [10, 25, 15]
print(list(zip(names, costs)))
price = dict(zip(names, costs))
print(price)
```

line 5 → `[('tea', 10), ('coffee', 25), ('samosa', 15)]`

line 7 → `{'tea': 10, 'coffee': 25, 'samosa': 15}`

`{}` is an empty dictionary (an empty set is `set()`, the sets sheet). `zip` (the iterables sheet) pairs two lists item by item, and `dict` turns the pairs into the same dictionary as the braces in panel 1. Python prints strings in single quotes, in the order the pairs went in.

3 `d[key]` looks up; `get` has a fallback

lookup.py

```
price = {"tea": 10, "coffee": 25}
print(price["coffee"])
print("chai" in price)
print(price.get("chai"))
print(price.get("chai", 0))
print(price["chai"])
```

output

```
25
False
None
0
stops: a KeyError
```

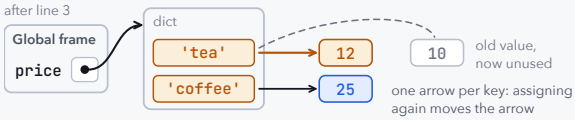
```
Traceback (most recent call last):
  File ".../lookup.py", line 6, in <module>
    print(price["chai"])
          ~~~~~^^^^^^^^
KeyError: 'chai'
```

`d[key]` gives the value, or stops with a **KeyError** that names the missing key. `key in d` asks whether the key is there. The **method** `d.get(key)` gives `None` for a missing key instead of an error; `d.get(key, 0)` gives the **default** you chose. Neither changes the dictionary.

4 Assignment adds a pair or replaces a value

line	code	price after the line	
1	`price = {"tea": 10}`	`{'tea': 10}`	
2	`price["coffee"] = 25`	`{'tea': 10, 'coffee': 25}`	new key: added
3	`price["tea"] = 12`	`{'tea': 12, 'coffee': 25}`	old key: replaced
4	`x = price.pop("tea")`	`{'coffee': 25}`	x = 12
5	`del price["coffee"]`	`{}`	removed

after line 3



`d[key] = value` adds the pair if the key is new, else replaces the value: keys are **unique**, so `{"tea": 10, "tea": 12}` gives `{'tea': 12}`. `d.pop(key)` removes a pair and returns its value; `del d[key]` just removes it. A missing key raises a KeyError in both, but `d.pop(key, 0)` gives 0.

5 Keys must be hashable

key tried	type	d = {key: 1}; print(d) gives	
`7`	int	`{7: 1}`	●
`"tea"`	str	`{'tea': 1}`	●
`(2, 3)`	tuple	`{(2, 3): 1}`	●
`[2, 3]`	list	`TypeError: unhashable type: 'list'`	■
`{"a": 1}`	dict	`TypeError: unhashable type: 'dict'`	■

```
grid = {(0, 0): "start", (2, 3): "goal"}
print(grid[(2, 3)]) → goal
sizes = {"tea": [10, 15]}
sizes["tea"].append(20) then print(sizes) → {'tea': [10, 15, 20]}
```

A key must be **hashable** (the sets sheet): Python turns it into a number, its hash, that says where to look, so a key must never change. Numbers, strings and tuples work; a list, set or dict can change, so it is a **TypeError**. A tuple key can name a grid cell. A value can be anything; a list value can grow in place.

6 Loop over keys, `values()` or `items()`

```
price = {"tea": 10, "coffee": 25, "samosa": 15}
print(list(price)) → ['tea', 'coffee', 'samosa']
print(list(price.values())) → [10, 25, 15]
print(price.items())
for item, cost in price.items():
    print(item, "costs", cost)
```

line 4 → `dict_items([('tea', 10), ('coffee', 25), ('samosa', 15)])`

pass	item	cost	printed
1	tea	10	tea costs 10
2	coffee	25	coffee costs 25
3	samosa	15	samosa costs 15

each pass unpacks one (key, value) tuple, in the order the keys went in

A `for` loop over a dictionary gives its keys, as `list(d)` does; `d.items()` gives `(key, value)` tuples to unpack (the iterables sheet). A dictionary keeps **insertion order** (Python 3.7 on): add `"chai"`, delete `"tea"`, add it back, and the keys are `['coffee', 'samosa', 'chai', 'tea']`.

Count every word, then rank the counts

The notebook's program on a text of our own, traced word by word, written with `get`, and sorted by count.

7 The word-frequency program, run for real

word_freq1.py, from the notebook (take(n, ...) written as [:n])

```
1  def word_freq(text):
2      def strip_punctuation(s):
3          return s.strip(".,?!\"\':;()[]{}")
4      freq = {}
5      for line in text.split("\n"):
6          for word in map(str.lower, map(strip_punctuation, line.split())):
7              if word in freq:
8                  freq[word] = freq[word] + 1
9              else:
10                 freq[word] = 1
11     return freq
12
13 import sys
14 f = open(sys.argv[1], "r")
15 n = int(sys.argv[2])
16 ffreq = word_freq(f.read())
17 def val(kv):
18     return kv[1]
19 for word, freq in sorted(ffreq.items(), key=val, reverse=True)[:n]:
20     print(f"{word} ⇒ {freq}")
```

story.txt (our own text: 33 words, 15 different)

```
1 The cat sat on the mat.
2 The dog sat on the log.
3 Did the cat see the dog?
4 Yes, the cat saw the dog!
5 The dog ran, and the cat ran after it.
```

terminal

```
$ python3 word_freq1.py story.txt 5
the ⇒ 10
cat ⇒ 4
dog ⇒ 4
sat ⇒ 2
on ⇒ 2
```

here sys.argv is the list `['word_freq1.py', 'story.txt', '5']`

lines 1 to 11: count the words
lines 13 to 16: read the file named on the command line
lines 17 to 20: sort by count, print the top n

A word's **frequency** is how many times it appears. The notes' plan: start with an empty dictionary; for each word, add 1 to its count if it is already a key, else add it with count 1. `strip_punctuation` trims both ends (`mat.` becomes `mat`); `str.lower` is `lower` as a plain function, for `map`. `sys.argv` holds the words typed after `python3`; `f.read()` gives the file as one string (the notes leave files to the Python tutorial).

8 Trace line 1; `get` does lines 7 to 10 in one

pass	split	word	in freq?	freq[word]	len(freq)	
1	The	'the'	False	1	1	new key
2	cat	'cat'	False	1	2	new key
3	sat	'sat'	False	1	3	new key
4	on	'on'	False	1	4	new key
5	the	'the'	True	2	4	+1
6	mat.	'mat'	False	1	5	new key

lines 7 to 10 of the program

```
if word in freq:
    freq[word] = freq[word] + 1
else:
    freq[word] = 1
```

the same job with get

```
freq[word] =
    freq.get(word, 0) + 1
```

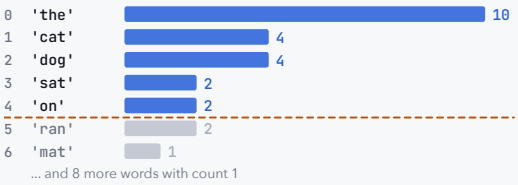
the same dict for the whole story: True

The fifth word, `the`, is already a key, so its count goes to 2 and `len(freq)` stays 4. `freq.get(word, 0) + 1` gives 1 for a new word, else the old count plus 1.

9 Sort the pairs by count, keep the top 5

```
def val(kv):    val(("the", 10)) → 10
    return kv[1]
top = sorted(freq.items(), key=val, reverse=True)
print(top[:5])
```

top: the (word, count) pairs, biggest count first



... and 8 more words with count 1

`key=val` makes `sorted` compare pairs by count (the functions are values sheet), `reverse=True` puts the biggest first, `[:5]` keeps five. `key=lambda kv: kv[1]` works too; `sorted(freq)` sorts words A to Z.

10 Common mistakes, with `price = {"tea": 10, "coffee": 25, "samosa": 15}` and `counts = {}`

you wrote	the last line Python prints, or what you see	why, and the fix
`print(price[0])`	`KeyError: 0`	keys, not positions
`counts["tea"] += 1`	`KeyError: 'tea'`	no count yet: use `get`
`print(10 in price)`	`prints False`	use `in price.values()`
`print(price.items()[0])`	`TypeError: 'dict_items' object is not subscriptable`	wrap it in `list()` first
`for k in price: del price[k]`	`RuntimeError: dictionary changed size during iteration`	loop over `list(price)`

11 Try it (answers in green)

1. `d = {"a": 1}; d["a"] += 2; print(d)`?
`{'a': 3}`

2. `len({"x": 1, "x": 2})`?
`1`: one pair per key.

3. `d = {}; print(d.get("b", 5), d)`?
`5 {}`: `get` adds nothing.

4. Count the letters of `"banana"` with `get`.
`{'b': 1, 'a': 3, 'n': 2}`

5. Group `["an", "be", "ax"]` by first letter.
`{'a': ['an', 'ax'], 'b': ['be']}`

6. `v = {"x": 3, "y": 7, "z": 5}; max(v, key=v.get)`?
`'y'`: the key with the top value.

unique, hashable keys · `d[k]` or `d.get(k, default)` · `in` checks keys · `items()` gives pairs · sort pairs with `key=`

Source: the word-frequency program (word_freq1.py) and its handwritten notes, nipunbatra.github.io/comp22; its more_itertools.take(n, ...) is written as the slice [:n], with the same result. The story and other examples are our own; every output and error comes from running the code with Python 3.11.11. Next: the putting it together sheet.